

EDD TRANSIENT



Max-Planck-Institut
für Radioastronomie

Development Team

Sachin Pradeep Etakkpepravan Thulicheri

Niclas Esser

Dr. Ewan Barr

Dr. Yunpeng Men

Department of Electronics, Software Development

Max Planck Institute for Radio Astronomy

January 11, 2025

Introduction

Fast transient astronomy holds the key to understanding processes in the universe at micro to millisecond timescales. The main population of such events in radio, including Fast Radio Bursts (FRBs), Pulsars, Rotating Radio Transients (RRATs), and long-period transients, remains largely undiscovered in extreme, unexplored parameter spaces. These phenomena provide insights into highly energetic events, such as mergers of stellar remnants and emissions from neutron stars like Pulsars and Magnetars. Increasing the number of observed transients enhances statistical power for studying the transient universe. Here, we present EDD-Transient, a commensal transient search pipeline for EDD backends.

Pipeline Architecture

The pipeline begins by receiving packetized data from the Alveo-Polyphase Filter Bank (PFB) using MKRECV, which implements the SPEED protocol. The captured voltage data is converted to intensity using `fft_det_acc`, followed by dedispersion using `dbdedispdb`, a TransientX-based dedispersion program. The dedispersed time series is then searched for single pulses using `dbsearch`. The flow of data between components is managed using PSRDADA ring buffers. A detailed explanation of each component is provided below, and the overall workflow is illustrated in the flowchart (Fig. 1).

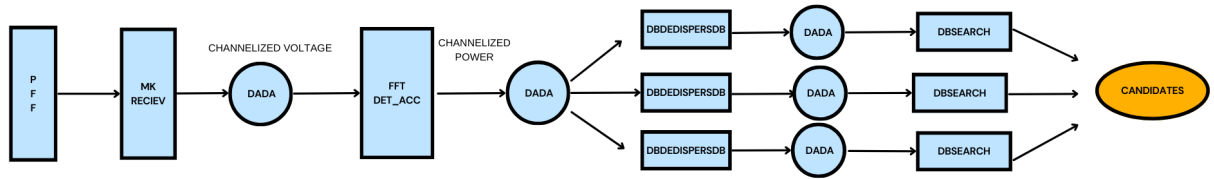


Figure 1: Flow chart of EDD Transient

MKRECV

- **Role:** Captures the packetized incoming voltage data from the telescope.
- **Key Functionality:**
 - Interfaces with the telescope’s channelized data stream from the PFB.
 - Processes input data in the format $T_{\text{out}} F T_{\text{in}} P$:
 - * Outer time dimension (T_{out})
 - * Coarse frequency channels (F)
 - * Fixed inner time bins (T_{in} - 2048 bins)
 - * Two polarizations (P - X and Y)

The input data type is `int8` complex.

- Stores the captured data in a shared buffer called `fftacc_input_buffer`. The buffer size is kept close to 1 GB by adjusting the size of the outer time dimension. This buffer connects to `fft_det_acc`, a customized voltage-to-intensity converter.

Detection Accumulator application - `fft_det_acc`

- **Role:** Converts raw voltage data into intensity.
- **Key Functionality:**
 - Applies FFT-based processing on the inner time dimension to increase the number of frequency channels and performs integration over the remaining bins of inner time dimension.
 - Supports three modes of operation:
 - * **f8n256:** FFT size of 8 per coarse channel with 256 integrations.
 - * **f16n128:** FFT size of 16 per coarse channel with 128 integrations.
 - * **f32n64:** FFT size of 32 per coarse channel with 64 integrations.
 - Allows additional integration via an accumulation parameter **nacc**. For example, with **mode=f16n128** and **nacc=6**, the effective integrated samples becomes 768.
 - Outputs data in the format TF , where the new frequency dimension size equals the product of coarse channels and FFT points (Ff), and the new outer time dimension size is
$$\frac{T_{\text{out}} \times 2048}{\text{FFT size} \times \text{nacc}}.$$
 - The output data is stored in another shared data buffer called `fftacc_output_buffer` in bytes format.

Dedispersion application (`dbdedispdb`)

- **Role:** `dbdedispdb` is a real-time dedispersion program based on TransientX.
- **Key Functionality:**
 - Cleans filterbank format data using advanced RFI mitigation tools provided by TransientX.
 - Applies a dedispersion algorithm across a range of dispersion measures (DMs).
 - Creates two shared data buffers:
 - * **DM-T buffer:** Contains the time series of the pulse for each DM trial, integrated over the frequency axis.
 - * **DM-T-subband buffer:** Contains a three-dimensional matrix where the DM-T array is generated by integrating only over a smaller sub-band of the total frequency range, preserving an additional frequency axis. This allows the creation of dynamic spectrum plots for each single-pulse candidate.

Candidate searching and clustering application (`dbsearch`)

- **Role:** Searches for single-pulse candidates using matched filters over a range of pulse widths and clusters the candidates to identify true signals.
- **Key Functionality:**
 - Identifies single-pulse signals in dedispersed time series.
 - Implements the DBSCAN clustering algorithm to filter out false candidates and detect true candidates.
 - During an observation, writes single-pulse candidates into a `.cands` file and generates dynamic spectra for each candidate.

Real-Time Candidate Plotter

In addition to the main components, the pipeline includes a feedback feature that visualizes candidates identified by `dbsearch` in the DM-T plane. This functionality is implemented by the `Real_time_plot_DMT.py` script, which periodically reads the `.cands` file at user-defined intervals. Based on the contents of this file, the script generates a DM-T plot, visually representing the distribution of candidates across the dispersion measure (DM) and time dimensions. The generated plot is then encoded in Base64 format and passed into the transient search pipeline, allowing seamless integration. The encoded plots are subsequently picked up by a Grafana monitoring interface, enabling real-time monitoring and visualization of transient candidates during pipeline operation.

Archiver

The pipeline includes an archiving module to efficiently manage the storage and organization of output data. This module ensures that dynamic spectrum files, such as *"candidate.png"*, are systematically compressed and archived. Every 10 seconds, the archiver compresses newly generated files, ensuring that each file has been written at least 5 seconds before the compression time to avoid conflicts. Once compressed, the files are added to a tarball archive, and both the uncompressed and compressed versions of the file are removed from the folder to save space. At the end of the observation, the archiver performs a final step, compressing and archiving any remaining files to ensure all data is preserved in a structured format.

Usage and configuration

The pipeline follows the EDD state model that has four stages in a measuring pipeline, they are pipeline configure, measurement prepare, measurement start, measurement stop and pipeline deconfigure. The stages are to be executed in order for every scan.

Configure

At the configuration stage, the pipeline receives critical information about the input stream, including:

- **Frequency bands of observation, bandwidths, central frequencies, and sampling rate:** The Effelsberg Radio Telescope observes across 8 bands, each containing 8 coarse frequency channels. The `Band_info` class is used to compute the number of coarse frequency channels, central frequency and bandwidth of the observation from the input stream parameters.
- **Continuity of frequency bands:** The pipeline performs a single-pulse search only if the frequency bands are continuous.

The user-controlled parameters for the `fft_det_acc` module are:

- **mode:** A string parameter with a default value of `'f16n128'`.
- **nacc:** An integer parameter with a default value of 4, which controls additional time scrunching of the data.

Once all these configurations are received the `fft_det_acc` input and output buffer sizes are calculated and are created. Furthermore the output buffers of `dbdedispdb` are instantiated at this stage with 1 byte memory. These buffers are then attached to the respective modules at measurement prepare.

Measurement Prepare

At the measurement preparation stage, two pipeline applications are configured and started in the order `fft_det_acc` → `dbdedispdb`. These modules require their configurations to be provided in the form of a JSON file. For `dbdedispdb`, the dedispersion configuration is stored as a nested dictionary under the key: “**dedisp**” and includes the following components:

1. “**preprocesslite**”: Parameters for:
 - Time and frequency scrunching (“**td**” and “**fd**”).
 - Zapping of outlier frequency channels using skewness and kurtosis (“**zaphre**”).
2. “**downsample**”: Optional further downsampling in:
 - Time (“**td**”).
 - Frequency (“**fd**”).
3. “**baseline**”: Baseline removal using a running median filter with:
 - “**width**”: Width of the filter in seconds.
4. “**rffi**”: RFI mitigation configuration including:
 - “**rfflist**”: Filters such as “**zdot**” (zero DM filter), “**kadaneF**” and “**kadaneT**” for frequency and time domain filtering.
 - “**zaplist**”: A list of channels to be zapped (floating-point values).
 - Thresholds for masking and Kadane filters: “**thremask**”, “**thrKadaneF**”, and “**thrKadaneT**” (measured in S/N).
5. “**subdedispersion**”: Configuration for dedispersion trials:
 - “**dms**”: Starting DM.
 - “**ddm**”: Interval between DM trials.
 - “**ndm**”: Number of DM trials.

In addition we also pass the configuration for `dbsearch` in this stage. The nested dictionary under “**dbsearch**” contains the following configuration components:

1. “**boxcar**”: Defines the boxcar width parameters for pulse detection:
 - “**minw**”: Minimum width of the pulse.
 - “**maxw**”: Maximum width of the pulse.
 - “**snrloss**”: The signal-to-noise ratio (S/N) loss allowed while applying the boxcar filter.
2. “**clustering**”: Configures parameters for candidate clustering in the DM-T plane. The true candidate is identified as the core point with the highest S/N. Key parameters include:
 - “**thre**”: S/N threshold for identifying candidates.
 - “**radius**”: Radius of the cluster in bins.
 - “**minpts**”: Minimum number of candidates required to form a cluster.
3. “**candplot**”: Configures the naming convention for the .png files of candidates produced after clustering:
 - “**rootname**”: Root name for the candidate plots. The default value is “**CAND**”.

At this stage, the archiver is also initiated. Archiving configuration parameters are extracted directly from the `"dbsearch"` configuration.

For further details on dedispersion, searching techniques, and parameter usage, refer to the following resources:

- TransientX GitHub repository.
- Fundi TransientX tutorial.

Measurement Start

At the measurement start, the MKRECEV network capture is initiated, and data streaming begins. The `dbsearch` module requires two data buffers to attach. Since these buffers are created by `dbdedispdb` only after reading the data, the `dbsearch` module is started only after the data streaming has commenced.

Measurement Stop

At the measurement stop, all components are stopped sequentially, beginning with the input stream and ending with `dbsearch`. The data buffers for `fft_det_acc` are reset at this point. These buffers are not deleted, allowing the user to return to the measurement prepare stage and modify the dedispersion and search configurations without affecting the configuration of `fft_det_acc`.

In contrast, the buffers created by `dbdedispdb` are dependent on the dedispersion configuration and are therefore deleted during this step. Additionally, every measurement stop call sends a stop flag to the archiver. The archiver performs a final compression and adds the compressed files to the tarball.

If the user chooses to revisit the measurement prepare stage within the same scan, the outputs for each measurement will be saved as separate tar files, with the UTC timestamp of the measurement start included in the file name.

Deconfigure

Deconfigure is a fail safe and terminates all the modules and buffers that are still running.

Sensors and real-time output

The pipeline monitors the input stream using a watchdog, which ensures that the pipeline proceeds only when data is being written to its input buffer. To identify any bottlenecks caused by slow pipeline components, sensors are used to monitor the input and output buffers of `fft_det_acc`. These sensor readings are displayed in the Grafana monitoring interface as input and output buffer statuses. If the `fft_det_acc` application is not performing optimally, the input buffer will become full. Conversely, if the dedispersion or search applications introduce delays, the output buffers will be impacted. Additionally, the real-time candidate plotter, as previously mentioned, updates every 10 seconds and displays the candidates in the monitoring interface.

Scalability and Future

The pipeline is designed to scale independently, allowing efficient exploration of the dispersion measure (DM) and pulse width parameter spaces, provided sufficient computing resources are available.

The future of transient science with the Effelsberg radio telescope looks particularly promising with the anticipated installation of phased array feed (PAF) receivers. These advanced receivers

are expected to generate up to a hundred beams on the sky, and the EDD Transient pipeline can be expanded to perform transient searches in each beam simultaneously.

A logical next step in this progression would be the development of a transient buffer dump that subscribes to the baseband recorder pipeline.